

# 1. Generative Adversarial Nets

기존의 생성 모델들은 확률분포를 직접 계산하거나 근사 추론, Markov Chain과 같은 복잡한 과정이 필요한 경우가 많았다. GAN에서는 이런 방식을 사용하지 않고 두 개의 신경망을 서로 경쟁시키는 방식으로 생성 모델을 학습한다.

GAN은 Generator와 Discriminator라는 두 개의 모델로 구성된다. Generator는 랜덤한 noise  $z$ 를 입력으로 받아 가짜 데이터를 생성하고, Discriminator는 입력된 데이터가 실제 데이터인지 Generator가 만든 가짜 데이터인지 구분한다. 논문에서는 Generator를 위조지폐를 만드는 사람, Discriminator를 위조지폐를 구별하는 경찰에 비유한다. Generator는 Discriminator를 속일 수 있도록 더 실제와 비슷한 데이터를 만들려고 하고, Discriminator는 실제 데이터와 가짜 데이터를 더 잘 구분하려고 한다. 이러한 경쟁을 반복하면서 Generator의 생성 능력이 좋아진다.

GAN의 핵심 목적함수는 다음과 같다.  $\min_D \max_G V(D, G) = E_{x \sim p_{data}} [\log D(x)] + E_{z \sim p_z} [\log (1 - D(G(z)))]$   $V(D, G) = E_{x \sim p_{data}} [\log D(x)] + E_{z \sim p_z} [\log (1 - D(G(z)))]$

Discriminator의 입장에서 보면 실제 데이터  $x$ 에 대해서는  $D(x)$ 가 1에 가까워야 하고, Generator가 만든  $G(z)$ 에 대해서는  $D(G(z))$ 가 0에 가까워야 한다. 반대로 Generator는 자신이 만든 데이터가 Discriminator에게 실제 데이터처럼 보이기를 원하기 때문에  $D(G(z))$ 가 1에 가까워지도록 학습한다. 즉 같은 목적함수를 두고 Generator와 Discriminator가 서로 반대 방향으로 학습하는 구조이다.

실제 학습에서는 Discriminator와 Generator를 번갈아 업데이트한다. 먼저 실제 데이터와 Generator가 만든 데이터를 이용해 Discriminator를 학습하고, 그 다음 Generator를 업데이트한다. 논문에서는 Discriminator를  $k$ 번 업데이트한 후 Generator를 한 번 업데이트하는 방식으로 설명하며 실험에서는  $k=1$ 을 사용한다.

또 하나 중요한 부분은 학습 초반의 gradient 문제이다. 학습 초반에는 Generator가 만드는 데이터의 품질이 좋지 않기 때문에 Discriminator가 가짜 데이터를 너무 쉽게 구분할 수 있다. 이 경우  $\log (1 - D(G(z)))$

를 최소화하는 원래 목적함수에서는 Generator가 충분한 gradient를 받지 못할 수 있다. 그래서 논문에서는 Generator가  $\log D(G(z))$

를 크게 만들도록 학습하는 방법을 사용할 수 있다고 설명한다. 최종적으로 도달하는 지점은 같지만 학습 초기에는 더 강한 gradient를 얻을 수 있다.

이론적으로 Generator가 고정되어 있을 때 최적의 Discriminator는 다음과 같이 표현된다.  $D^*(x) = \frac{p_{data}(x)}{p_{data}(x) + p_g(x)}$

그리고 Generator가 만드는 분포  $p_g$ 가 실제 데이터의 분포  $p_{data}$ 와 같아지면 Discriminator는 두 분포를 구별할 수 없게 된다. 이때 모든 데이터에 대해  $D(x) = 1/2$ 가 된다. 논문에서는 GAN의 목적함수를 Jensen-Shannon divergence와 연결하여  $p_g = p_{data}$ 일 때 전역 최적점이 된다는 것을 보인다.

다만 이 결과는 이상적인 상황을 가정한 이론적인 결과이다. 실제 GAN에서는 Generator와 Discriminator가 유한한 크기의 신경망으로 구성되고, 신경망의 parameter space에는 여러 critical point가 존재하기 때문에 실제 학습에서 항상 이론적인 최적점에 도달한다고 보장할 수는 없다. 논문에서도 이러한 차이를 언급한다.

실험은 MNIST, Toronto Face Database, CIFAR-10을 이용해 진행되었다. Generator와 Discriminator는 multilayer perceptron 기반으로 구성되었으며, 생성 결과와 log-likelihood 추정을 통해 모델을 평가했다. GAN의 경우 명시적인  $p_g(x)p_g(x)$ 를 계산할 수 없기 때문에 생성한 샘플에 Gaussian Parzen window를 적용하여 test set에 대한 likelihood를 추정하였다. 하지만 논문에서도 이 방법은 variance가 크고 고차원 데이터에서는 정확한 평가가 어렵다는 한계를 언급한다.

논문에 제시된 생성 결과를 보면 MNIST, 얼굴 이미지, CIFAR-10 이미지 등이 생성된다. 생성 이미지 옆에는 가장 가까운 training sample도 같이 보여주는데, Generator가 단순히 training data를 그대로 외워서 출력한 것이 아니라는 것을 확인하기 위한 것이다. 또한 latent space에서 두  $z$  값 사이를 interpolation했을 때 이미지가 자연스럽게 변화하는 결과도 보여준다.

GAN의 장점은 샘플을 생성할 때 Markov Chain이 필요하지 않고 Generator의 forward propagation만 수행하면 된다는 점이다. 또한 학습에는 일반적인 backpropagation을 사용할 수 있고, 다양한 형태의 differentiable function을 모델로 사용할 수 있다.

반면 단점도 있다. Generator가 만드는 분포  $p_g(x)p_g(x)$ 를 명시적으로 계산할 수 없으며, Generator와 Discriminator의 학습 균형을 맞추는 것이 중요하다. 논문에서는 Generator가 너무 많이 학습되어 여러 개의 서로 다른  $z$ 를 비슷한 하나의 결과로 보내게 되면 데이터의 다양성을 충분히 표현하지 못할 수 있다고 설명한다.

정리하면 GAN은 실제 데이터와 생성 데이터의 차이를 직접 계산하는 대신, 판별기인 Discriminator를 통해 학습 신호를 얻는 생성 모델이다. Generator와 Discriminator를 경쟁시키는 minimax game을 생성 모델 학습에 적용했다는 것이 이 논문의 가장 중요한 아이디어라고 볼 수 있다.